```
knowYourClassicMetal(...['album' => 'Master of Puppets', 'artist' =>
'Metallica',
   'year' => '1986', 'Elektra Records']);
```

```
Output:
Master of Puppets by Metallica, released in 1986 on Elektra Records
Master of Puppets by Metallica, released in 1986 on Elektra Records
Fatal error: Uncaught Error: Cannot use positional argument after named
argument during unpacking
```

The first two calls are fine. One has all named arguments and the other has the album title as the first array element (plus correct string keys for the rest). The third call omits the `'label'` key in the final array position and our program dies with an error.

And there we have it, all the weird and wonderful ways that functions can receive their arguments in PHP 8. Hopefully the lesson to take away here is that it's best to keep things simple. The more complexity in argument handling in your code, the more error prone it will be.

Functions and Scope

Now is an excellent time to introduce a very important concept in PHP (and programming in general), which is particularly important to functions, and that concept is *scope*. Scope (or more formally, *lexical scope*) can be generally defined as the part of a program where the association of identifiers with their data is valid. Throughout the code we've seen so far we have used variables, if you remember the chapter on data and variables, you'll know that the variables themselves are just references to data in memory. When PHP encounters a variable in an expression it will access the data and use it to evaluate the expression. Scope is essentially a set of simple rules that determine whether or not a particular variable can be used to access the intended data. (Note that although function names can be considered as identifiers, those are not subject to scoping rules.)